

Clase 4: Comunicar con R y Aplicaciones de uso de IA Generativa

Análisis de Datos y Estadística Computacional

Prof. Gabriel Duarte Zúñiga

Universidad Adolfo Ibáñez | DIAPP-2026

20 de junio de 2026

¡Bienvenid@s a la Clase 4!

Cierre de la Unidad I

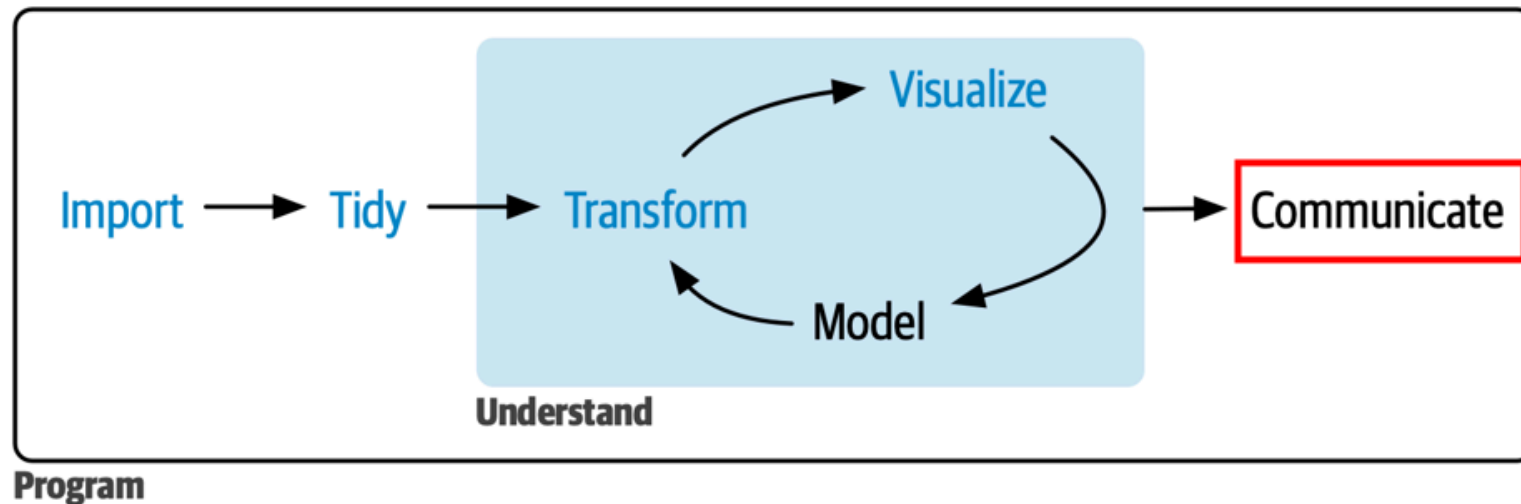
Esta es la **última clase** de la Unidad I: *Herramientas de Ciencia de Datos con R*. El objetivo es cerrar el ciclo de trabajo con datos aprendiendo a **comunicar** resultados de forma reproducible y a **acelerar** nuestro flujo de trabajo con herramientas de IA generativa.

Los contenidos específicos de la clase son:

- ▶ Principios básicos para contar historias con datos.
- ▶ Qué son R Markdown y Quarto, y por qué preferimos Quarto.
- ▶ Estructura de un documento Quarto: YAML, chunks y texto.
- ▶ Aplicaciones de IA generativa para programar en R.

Resumen Unidad I

Clase	Contenido
Clase 1	Importar datos: <code>read_csv</code> , <code>read_excel</code> , <code>read_dta</code> , <code>left_join</code>
Clase 2	Transformar datos: <code>dplyr</code> , <code>tidyr</code> , <code>lubridate</code> , <code>forcats</code>
Clase 3	Visualizar datos: <code>ggplot2</code> , gramática de gráficos, extensiones
Clase 4	Comunicar: documentos reproducibles + IA generativa



Contar historias con datos

¿Por qué contar una historia?

- ▶ La mayor parte de la visualización y el análisis de datos se realiza con el fin de **comunicar** algo a alguien ([Schimel 2012](#)).
- ▶ Una historia es un conjunto de observaciones o hechos presentados en un orden que genera una **reacción** en la audiencia: curiosidad, sorpresa, convicción.
- ▶ La clave es construir un *arco narrativo* claro: ir de la **tensión** (el problema o la pregunta) hacia la **resolución** (el hallazgo o la recomendación).

El marco SUCCES

Una buena historia con datos cumple seis principios ([Heath y Heath 2013](#)):

Principio	¿Qué significa?
S imple	El mensaje principal es claro y compacto
U nexpected	Presenta algo novedoso que genera curiosidad
C oncrete	Combina lo abstracto de las ideas con lo concreto de los datos
C redible	Se sustenta en datos y metodología robusta
E moional	Motiva a la audiencia: no solo informa, sino que genera conexión
S tores	Tiene una estructura interna coherente

Un plan para construir una historia con datos

A continuación, se propone un plan de acción para construir una historia con datos ([Alexander 2023](#))

1. **Planifiquen** el punto final: ¿qué quieren que la audiencia recuerde o haga?
2. **Explore**n los datos buscando patrones y hallazgos relevantes.
3. **Diseñen** las visualizaciones que mejor comunican esos hallazgos.
4. **Estructure**n la narrativa: contexto → hallazgo → implicancia.
5. **Compartan** usando un formato reproducible (Quarto, R Markdown).



Tip

Piensen en el gráfico como el **punto focal** de la historia: si alguien ve solo el gráfico sin leer el texto, ¿entiende el mensaje principal? ([NUSSBAUMER KNAFLIC 2025](#))

Documentos reproducibles

¿Por qué usar Quarto o R Markdown?

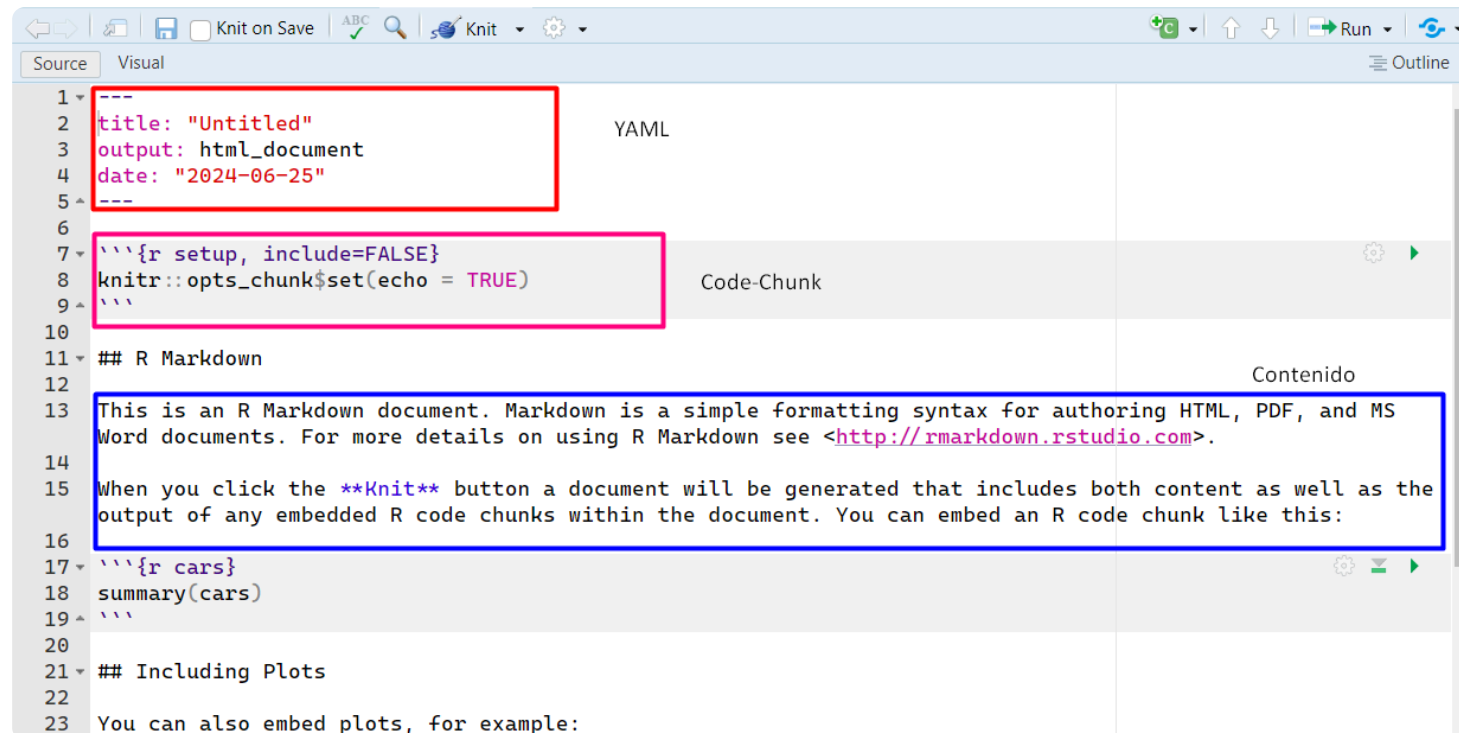
El flujo tradicional implica:

1. Analizar en R → copiar resultados → pegar en Word → formatear → exportar.
2. Si los datos cambian → **repetir todo desde el paso 1.**

R Markdown y **Quarto** resuelven esto: integran código, resultados y texto en **un solo archivo** que se compila automáticamente.

¿Qué es R Markdown?

- ▶ Entorno para crear documentos reproducibles que combinan **código R**, **resultados** (tablas, gráficos) y **texto narrativo** en un solo archivo **.Rmd**.
- ▶ Permite compilar (*knit*) a múltiples formatos: HTML, PDF, Word, PowerPoint.
- ▶ Fue creado por Yihui Xie (2014) y es mantenido por Posit (ex RStudio).



```
1 ---
2 title: "Untitled"
3 output: html_document
4 date: "2024-06-25"
5 ---
6
7 ```{r setup, include=FALSE}
8 knitr::opts_chunk$set(echo = TRUE)
9 ```
10
11 ## R Markdown
12
13 This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS
14 Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.
15
16 When you click the Knit button a document will be generated that includes both content as well as the
17 output of any embedded R code chunks within the document. You can embed an R code chunk like this:
18
19 ```{r cars}
20 summary(cars)
21 ```
22
23 ## Including Plots
24
25 You can also embed plots, for example:
```

¿Qué es Quarto?

- ▶ **Quarto** (`.qmd`) es el sucesor de R Markdown: un sistema de publicación científica de **código abierto** construido sobre Pandoc ([«Quarto», s. f.](#)).
- ▶ Ventajas sobre R Markdown:
 - **Multilingüe**: funciona con R, Python y Julia.
 - **Todo en uno**: no necesita paquetes adicionales (bookdown, blogdown, xaringan) — todo está integrado.
 - **Herramientas modernas**: callouts, paneles con tabs, cross- references, layout avanzado, interactividad.
 - **Independiente de RStudio**: funciona desde VS Code, terminal, Jupyter.

Características de cada uno

	R Markdown (.Rmd)	Quarto (.qmd)
Lenguaje	Solo R	R, Python, Julia, Observable
Ecosistema	Muchos paquetes separados	Todo integrado
Funcionalidades	Depende del paquete	Callouts, tabs, layouts nativos
Aprendizaje	Amplia comunidad, mucha documentación	Documentación excelente, creciendo
Recomendación de uso	Proyectos existentes en .Rmd	Proyectos nuevos



Tip

Para este curso y la **Tarea 1** podremos usar **Quarto** (**.qmd**) o **R Markdown** (.Rmd). La sintaxis es casi idéntica a R Markdown, por lo que si ya saben **.Rmd**, la transición es inmediata.

Estructura de un documento Quarto

Un archivo `.qmd` tiene **tres componentes**:

```
---
title: "ggplot2 demo"
author: "Norah Jones"
date: "5/22/2021"
format:
  html:
    fig-width: 8
    fig-height: 4
    code-fold: true
---
```

```
## Air Quality
```

```
@fig-airquality further explores the impact of temperature on
ozone level.
```

```
{r}
#| label: fig-airquality
#| fig-cap: "Temperature and ozone level."
#| warning: false

library(ggplot2)
ggplot(airquality, aes(Temp, Ozone)) +
  geom_point() +
  geom_smooth(method = "loess")
}
```

ggplot2 demo

Norah Jones

May 22nd, 2021

Air Quality

[Figure 1](#) further explores the impact of temperature on ozone level.

► Code

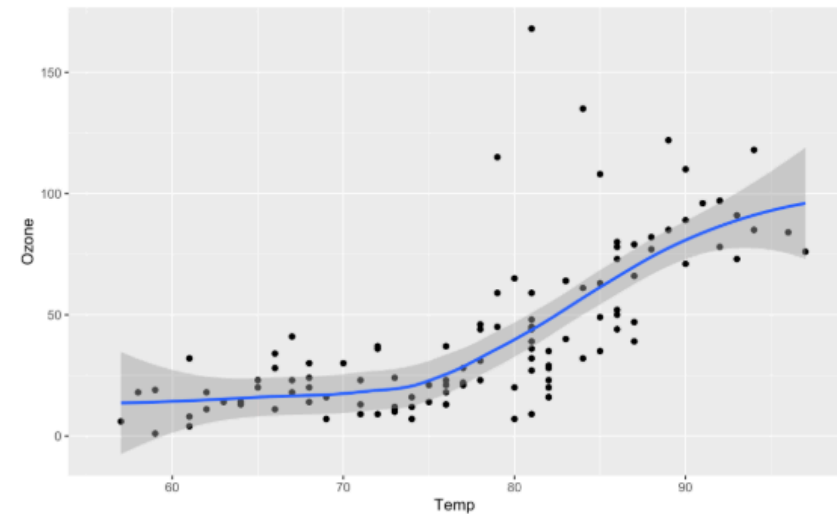


Figure 1: Temperature and ozone level.

1. YAML (*Yet Another Markup Language*)

- ▶ Va al **inicio** del documento, delimitado por `---`.
- ▶ Define **metadatos** y opciones de configuración: título, autor, fecha, formato de salida, tabla de contenidos, tema, etc.

```
1 ---
2 title: "Análisis de Homicidios en la RM"
3 author: "Gabriel Duarte"
4 date: "2026-06-20"
5 format:
6   html:
7     toc: true
8     theme: flatly
9     code-fold: true
10 execute:
11   echo: true
12   warning: false
13 ---
```

2. Chunks de código

- ▶ Bloques delimitados por ```{r}` y ````` que contienen código R ejecutable.
- ▶ Atajo para insertar: **Ctrl + Alt + I** (Mac: **Cmd + Option + I**).
- ▶ Se configuran con opciones `#|` al inicio del chunk:

```
1  ``{r}
2  #| echo: true
3  #| eval: true
4  #| warning: false
5  #| fig-width: 8
6
7  library(ggplot2)
8  ggplot(mtcars, aes(x = wt, y = mpg)) +
9    geom_point()
10 ``
```


Opciones más comunes de los chunks

Opción	Efecto
<code>echo: true/false</code>	¿Muestra el código en el documento?
<code>eval: true/false</code>	¿Ejecuta el código?
<code>warning: false</code>	Ocultar warnings
<code>message: false</code>	Ocultar mensajes
<code>include: false</code>	Ejecuta pero no muestra ni código ni resultado
<code>fig-width / fig-height</code>	Tamaño del gráfico
<code>code-fold: true</code>	Código colapsable (solo HTML)

3. Texto en Markdown

- Todo lo que no es YAML ni chunk es **texto narrativo** escrito en Markdown:

Sintaxis	Resultado
# Título	Encabezado nivel 1
## Subtítulo	Encabezado nivel 2
negrita	negrita
<i>*cursiva*</i>	<i>cursiva</i>
`código`	<code>código</code>
[texto](url)	Hipervínculo
	Imagen

Compilar (*Render*)

- ▶ Para generar el documento final: **Ctrl + Shift + K** (Mac: **Cmd + Shift + K**), o hacer clic en el botón **Render**.
- ▶ Quarto convierte el `.qmd` → ejecuta el código → genera el HTML/PDF/Word.

Advertencia

Para generar documentos **PDF** se necesita una distribución de LaTeX. La más sencilla es TinyTeX:

```
1 install.packages("tinytex")
2 tinytex::install_tinytex()
```

Demostración en vivo



Demo: Crear un documento Quarto

1. **File** → **New File** → **Quarto Document**
2. Elegir título, autor y formato (HTML)
3. Explorar la estructura: YAML, chunks, texto
4. Insertar un chunk con un gráfico (**Ctrl + Alt + I**)
5. Compilar con **Render** y ver el resultado

(También veremos cómo crear un archivo R Markdown para comparar la diferencia)

Recursos útiles

- ▶ [Quarto — Get Started](#) — guía oficial de instalación y primeros pasos.
- ▶ [R for Data Science, Cap. 28: Quarto](#) — el libro de referencia del curso (Wickham & Grolemund).
- ▶ [R Markdown: The Definitive Guide](#) — referencia completa de R Markdown (Xie et al., 2018).
- ▶ [Quarto Cheat Sheet](#) — resumen de una página.
- ▶ [Get started with Quarto — Mine Çetinkaya-Rundel](#) — tutorial en video (recomendado).

IA Generativa para programar en R

¿Qué es la IA generativa?

- ▶ Los modelos de lenguaje de gran escala (LLMs) como **ChatGPT** (OpenAI), **Claude** (Anthropic), **Gemini** (Google) y **Copilot** (Microsoft) pueden generar código, explicar funciones y asistir en el análisis de datos.
- ▶ Son especialmente útiles para:
 - **Acelerar** tareas repetitivas.
 - **Aprender** funciones o paquetes nuevos.
 - **Depurar** errores en el código.

Precaución

La IA no reemplaza el **juicio humano**: siempre verifiquen que el código generado sea correcto y que los resultados tengan sentido. Entiendan la IA como una **extensión** de sus capacidades, no un sustituto.

Herramientas de acceso gratuito

Herramienta	Acceso	Ventaja principal
Google Gemini	Gratuito con cuenta Google	Integración con Google Colab para ejecutar R
ChatGPT	Gratuito (GPT-4o limitado)	Amplia comunidad, muchos ejemplos
Claude	Gratuito (limitado)	Respuestas detalladas, buen razonamiento



Tip

Para este curso recomendamos **Google Gemini** por su acceso gratuito y la posibilidad de ejecutar código R directamente en **Google Colab**.

Ingeniería de prompts: la clave

Un **prompt** es la instrucción que le damos a la IA. La calidad de la respuesta depende directamente de la calidad del prompt.

Un buen prompt tiene **4 componentes** ([White et al., s. f.](#)):

1. **Rol**: quién es la IA en este contexto.
2. **Contexto**: qué datos tenemos, qué paquetes usamos.
3. **Tarea**: qué queremos que haga, de forma específica.
4. **Formato**: cómo queremos la respuesta.

Estructura de un buen prompt

Prompt vago ✗

“Hazme un gráfico de barras en R”

→ La IA no sabe qué datos usar, qué variable graficar ni qué estilo aplicar.

Prompt estructurado ✓

“Usando tidyverse y el dataset gapminder en R, crea un gráfico de barras horizontales que muestre la esperanza de vida promedio por continente en 2007. Usa theme_minimal(), colores de la paleta Set2 de RColorBrewer, y ordena las barras de mayor a menor. Agrega etiquetas con el valor sobre cada barra.”

Anatomía del prompt estructurado

[ROL] Eres un experto en R y ggplot2.

[CONTEXTO] Estoy trabajando con el dataset ``gapminder`` del paquete `gapminder` en R. Filtré los datos para el año 2007 y calculé la esperanza de vida promedio por continente.

[TAREA] Necesito un gráfico de barras horizontales que muestre la esperanza de vida promedio por continente, ordenado de mayor a menor, con etiquetas numéricas sobre cada barra.

[FORMATO] Dame solo el código en R usando `tidyverse` y `ggplot2`, con comentarios breves en español explicando cada paso.

Consejo práctico

Cuanto más contexto le den (qué paquetes usan, qué estructura tienen los datos, qué estilo visual prefieren), mejor será el resultado. Si el primer resultado no es perfecto, **iterar** corrigiendo el prompt.

Aplicaciones concretas

Aplicación 1: Explicar código

Cuando encuentren código que no entienden, pueden pedirle a la IA que lo explique línea por línea:

Explícame qué hace este código de R línea por línea, en español y de forma sencilla para alguien que está aprendiendo:

```
gapminder |>  
  filter(year == 2007) |>  
  group_by(continent) |>  
  summarise(media = mean(lifeExp)) |>  
  ggplot(aes(x = reorder(continent, media), y = media)) +  
  geom_col(fill = "steelblue") +  
  coord_flip() +  
  theme_minimal()
```



Tip

Este uso es especialmente valioso para **aprender**: en vez de buscar cada función por separado, obtienen una explicación contextualizada del flujo completo.

Aplicación 2: Buscar funciones o generar código

Cuando no saben qué función usar para resolver un problema:

Usando R y tidyverse, necesito leer un archivo Excel que tiene las primeras 3 filas con encabezados combinados y los datos reales comienzan en la fila 4. El archivo tiene datos hasta la fila 350 y después hay filas de totales que no quiero incluir.
¿Qué función y argumentos debo usar?

O para generar código desde una descripción:

Tengo un data frame llamado `homicidios` con columnas `comuna`, `anio` (2024 y 2025) y `n` (cantidad de homicidios). Necesito pivotear la tabla para que cada año sea una columna, calcular la diferencia entre 2025 y 2024, y ordenar de mayor a menor diferencia. Dame el código en R usando tidyverse.

Aplicación 3: Modificar un gráfico con **ggplot2**

Un uso muy práctico: partir de un gráfico base y pedirle mejoras específicas:

Tengo este gráfico en R:

```
ggplot(gap_2007, aes(x = lifeExp)) +  
  geom_histogram()
```

Necesito que le hagas las siguientes mejoras:

1. Cambiar el color de relleno a "steelblue" con bordes blancos
2. Usar 15 bins
3. Agregar título "Distribución de la esperanza de vida (2007)"
4. Poner etiquetas en español en los ejes
5. Usar `theme_clean()` del paquete `ggthemes`
6. Formato chileno en el eje y (punto separador de miles)



Tip

La ventaja de este enfoque es que **ustedes mantienen el control**: parten de código que ya entienden y le piden mejoras incrementales, verificando cada resultado.

Ejemplo en vivo con Gemini en Google Colab



Demo: IA generativa en acción

1. Abrir [Google Colab](#) con R
2. Usar Gemini integrado para:
 - ▶ **Explicar** un bloque de código
 - ▶ **Generar** un gráfico paso a paso
 - ▶ **Iterar** pidiendo mejoras al gráfico
3. Copiar el resultado final a RStudio

Buenas prácticas con IA

- ▶ **Siempre verificar** el código generado: ejecutarlo, revisar el resultado, entender qué hace cada línea.
- ▶ **No copiar sin entender**: si no pueden explicar qué hace el código, no lo usen. La Tarea 1 requiere **justificación escrita**.
- ▶ **Iterar**: si la primera respuesta no es perfecta, refinan el prompt. La conversación es un ida y vuelta.
- ▶ **Citar**: si usaron IA para generar código, indíquenlo. La transparencia es parte de la integridad académica.
- ▶ **Aprender**: usen la IA como **tutor**, no como atajo. Pedir explicaciones es más valioso que pedir soluciones.



Pregunta N° 1

Un alumno le pide a la IA: *“Hazme la tarea 1 del curso”*. ¿Cuál es el principal problema con este prompt?

¿Qué falla?

Falta especificar el formato de salida

La IA no sabe programar en R

Falta contexto, datos y una tarea específica

Debería usar Python en vez de R

Revisar

Reiniciar

Cierre de la Unidad I

Tarea 1

- ▶ **Fecha de entrega:** 28 de junio de 2026.
- ▶ **Formato:** documento Quarto (`.qmd`) o R Markdown (`.Rmd`) compilado a **HTML**.
- ▶ **Contenido:** manejo, transformación y visualización de datos con R.
- ▶ **Entrega:** a través de **webcursos** en el buzón correspondiente.

! Recordatorio

La tarea es **individual**. El código debe estar completo y documentado con comentarios. Se permite herramientas de IA generativa solo como apoyo. Si usan funciones distintas a las vistas en clases o explicitadas, deben justificarlo. La IA **no** reemplaza su trabajo, solo lo puede potenciar y ayudar a entender mejor lo que hacen.

Próxima clase: Unidad II

A partir de la **Clase 5** comenzamos la Unidad II: *Estadística Aplicada con R*, donde aprenderemos a:

- ▶ Realizar **inferencia estadística**: intervalos de confianza y pruebas de hipótesis.
- ▶ Construir modelos de **regresión lineal**.
- ▶ Usar el paquete **infer** para inferencia tidy.

Referencia principal Unidad II

[ModernDive](#) — Ismay, Kim & Valdivia (2025). *Statistical Inference via Data Science: A ModernDive into R and the Tidyverse* (2ª ed.).

Recapitulación

Tema	Contenido clave
Comunicar	Marco SUCCES, arco narrativo, estructura contexto → hallazgo → implicancia
R Markdown	Archivo <code>.Rmd</code> , compilar con <i>knit</i> , múltiples formatos de salida
Quarto	Archivo <code>.qmd</code> , sucesor de R Markdown, multilingüe, todo integrado
Estructura	YAML (metadatos) + Chunks (código) + Texto (Markdown)
IA generativa	Prompt = Rol + Contexto + Tarea + Formato
Aplicaciones	Explicar código, buscar funciones, modificar gráficos

Bibliografía

- Alexander, Rohan. 2023. *Telling Stories with Data: With Applications in R*. Chapman y Hall/CRC Data Science Series. Milton: CRC Press LLC.
- Heath, Chip, y Dan Heath. 2013. *Made to stick: why some ideas survive and others die*. New York: Random House.
- NUSSBAUMER KNAFLIC, COLE. 2025. *STORYTELLING WITH DATA: a data visualization guide for business professionals*. S.I.: JOHN WILEY.
- «Quarto». s. f. <https://quarto.org/>.
- Schimmel, Joshua. 2012. *Writing science: how to write papers that get cited and proposals that get funded*. Oxford ; New York: Oxford University Press.
- White, Jules, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, y Douglas C. Schmidt. s. f. «A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT». <https://doi.org/10.48550/ARXIV.2302.11382>.
- Xie, Yihui, J. J. Allaire, y Garrett Grolemund. 2019. *R Markdown: the definitive guide*. Chapman & Hall/CRC, the R series. Boca Raton London New York: CRC Press, Taylor & Francis Group.